

Design Document APCSA

Undertale (Basic Game Overworld) in Processing

Made by Daniel and Yunus, Period 1.

→ About:

Our project is a recreation of the first part of the popular indie game Undertale. The main premise of the game is that you are a human who has ended up in the underworld, and you have to make it out by either killing or sparing. This is done in tandem with interacting with various NPCs and Monsters. We implemented the game's basic features, such as encountering the first dialogue, player animation, and hitbox detection. Everything is as similar to the original as we could make it.

→ How it works:

Undertale is a completely keyboard-based game, and therefore, we preserved the original keybinds of the game.

- Arrow Keys/WASD: Move the player in the overworld, move the soul in battle]
- Z key: Select, progress dialogue

Full list of functionalities/libraries (for more information, please see our more indepth printed copy)

- Battle System (Skeleton, proof of concept)
- Dialogue System
- Robust Graphics Handler, built from the ground up
- Player movement and animation in all 8 cardinal directions
- Room functionality
- Sound File Library -> works by loading .wav/.mp3 files, as PImage does for .pngs. Used for the dialogue speech sound for Flowey.
- Runnable Java Library -> Used for event scheduling, a customizable way to schedule certain events to happen at the end of the draw function to not interfere with key game processes.

Extra Section: Functionalities

Bold means that Daniel worked on it

Underline means that Yunus worked on it

→ Battle System

Turn-based battle system with bullet hell mixed in. None of the options are present in the tutorial battle, which is just dodging, so they will not be implemented in this rough project sketch.

- Player chooses an action
- The battle ends or continues based on the action taken
- They control the soul with the arrow keys to dodge bullet attacks

→ **Main Character Functionality**

- Controlling the main character Frisk with the up, down, left, and right arrow keys. Having working animation in all directions (2-frame animation), switching between animation frames
- Players are bound by the room bounds and obstacles

→ Room Functionality

- Entering rooms.
- Exiting rooms.
 - Both at the correct positions.
- Room moves based on player position (camera mode)
- The room you are in is tracked
- The Room class is used to define rooms

- Rooms contain hitboxes for collision, starting cutscenes, and defining the bounds of the world camera.

→ **Graphics Handler**

- Complete with :
 - a. Z-index ordering
 - b. Drawing every frame
 - c. Hitbox detection updating
 - d. Removing game objects that requested to remove themselves last frame
 - e. Updating the room
 - f. Player animation
 - g. A camera handler that changes the camera position based on the camera's bounds
 - h. Displaying dialogue
- An all-in-one handler that will update every game object in order of z-index, from least to greatest. every call of the draw() function.

→ **Dialogue**

- Contains a Text class and a PImage for the character portrait
- Running similar to a linked list, advanced when enter/z is pressed
- When all dialogue is exhausted, the dialogue box is removed.
- We will implement this with a class that holds multiple game objects, including:
 - Talking portrait that has animation
 - Text that scrolls and auto makes lines based on the number of characters
 - A rectangle that opens when dialogue is initiated

FINAL UML DIAGRAM AS OF JUNE 1st, 2026

https://lucid.app/lucidchart/d2e55014-6965-4119-9f13-c0862a2651b4/edit?view_items=d8FZqIVo9Lea&page=0_0&invitationId=inv_ae7cbde6-cf02-40ed-ad29-d81623415960

